

LABORATORIUM 7

# Formularze i walidacja danych

React Hook Form + Zod · Dostępność WCAG · 90 minut

|                                                   |                                               |                                 |                                     |
|---------------------------------------------------|-----------------------------------------------|---------------------------------|-------------------------------------|
| Kurs<br><b>Zaawansowany interfejs użytkownika</b> | Prowadzący<br><b>mgr inż. Dominik Aksamit</b> | Czas trwania<br><b>90 minut</b> | Semestr / rok<br><b>1 / 2025–26</b> |
|---------------------------------------------------|-----------------------------------------------|---------------------------------|-------------------------------------|

## 1. Cele zajęć

| WIEDZA                                                                             | UMIEJĘTNOŚCI                                                             | KOMPETENCJE SPOŁECZNE                                                |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------|
| Zna architekturę RHF (register, Controller, useFieldArray) i schemat walidacji Zod | Implementuje wieloetapowy formularz z persystencją danych między krokami | Stosuje standardy WCAG 2.1 AA w formularzach (ARIA, kontrast, focus) |
| Rozumie typowanie przez TypeScript z z.infer<typeof schema>                        | Tworzy reguły walidacji z komunikatami po polsku i obsługuje refine()    | Identyfikuje i uzasadnia unikanie anti-patterns w formularzach       |
| Wie, czym różni się walidacja kliencka od serwerowej                               | Obsługuje błędy serwera i mapuje je na pola formularza przez setError()  | Przeprowadza audit dostępności narzędziem Lighthouse / axe DevTools  |

## 2. Zadanie: wieloetapowy formularz rejestracji

Student implementuje formularz rejestracyjny podzielony na 3 kroki. Dane z kroków 1 i 2 są agregowane i wysyłane jednym żądaniem POST po potwierdzeniu w kroku 3. Każdy krok jest walidowany osobnym schematem Zod.

### Krok 1 – Dane osobowe

| Pole            | Typ    | Reguła walidacji (Zod)                                 |
|-----------------|--------|--------------------------------------------------------|
| firstName       | string | min(2, 'Imię musi mieć co najmniej 2 znaki')           |
| lastName        | string | min(2, 'Nazwisko musi mieć co najmniej 2 znaki')       |
| email           | string | email('Podaj poprawny adres e-mail')                   |
| password        | string | min(8) + regex([A-Z]) + regex([0-9])                   |
| confirmPassword | string | refine(data => data.password === data.confirmPassword) |

+ wskaźnik siły hasła: słabe / średnie / silne (renderowany poniżej pola password)

### Krok 2 – Preferencje

| Pole | Typ | Uwagi implementacyjne |
|------|-----|-----------------------|
|------|-----|-----------------------|

|                     |          |                                            |
|---------------------|----------|--------------------------------------------|
| categories          | string[] | useFieldArray, min 1 element wymagany      |
| notifications.email | boolean  | Controller + checkbox, aria-label wymagany |
| notifications.push  | boolean  | Controller + checkbox                      |
| newsletter          | boolean  | opcjonalne, opt-in                         |

### Krok 3 – Podsumowanie i potwierdzenie

- Widok read-only z danymi z kroków 1 i 2
- Checkbox RODO (required: true)
- Przycisk "Zarejestruj się" wyzwała handleSubmit
- Odpowiedź 409 (email zajęty) → setError('email') + powrót do kroku 1
- Odpowiedź 500 → setError('root.serverError', { message: 'Błąd serwera...' })

### Harmonogram 90 minut

| Czas      | Blok            | Zakres pracy                                                   |
|-----------|-----------------|----------------------------------------------------------------|
| 0–10 min  | Wprowadzenie    | RHF vs Formik, po co Zod, demonstracja działającego formularza |
| 10–25 min | Krok 1 – Dane   | useForm, zodResolver, register, wskaźnik siły hasła            |
| 25–45 min | Krok 2 – Prefsy | Controller, useFieldArray, walidacja tablicy                   |
| 45–65 min | Krok 3 – Submit | handleSubmit, setError, obsługa błędów serwera                 |
| 65–80 min | Dostępność      | aria-labels, focus management, Lighthouse audit                |
| 80–90 min | Podsumowanie    | Q&A, omówienie anti-patterns, ocena aktywności                 |

## 3. Instrukcja: React Hook Form + Zod

### Instalacja zależności:

```
npm install react-hook-form @hookform/resolvers zod
```

### 3.1 – Schema walidacji Zod (krok 1)

```
import { z } from 'zod';

export const step1Schema = z
  .object({
    firstName: z.string().min(2, 'Imie musi miec co najmniej 2 znaki'),
    email: z.string().email('Podaj poprawny adres e-mail'),
    password: z.string()
      .min(8, 'Haslo musi miec co najmniej 8 znakow')
      .regex(/[A-Z]/, 'Haslo musi zawierac wielka litere')
      .regex(/[0-9]/, 'Haslo musi zawierac cyfre'),
    confirmPassword: z.string(),
  })
  .refine(
    (data) => data.password === data.confirmPassword,
    { message: 'Hasla musza byc identyczne', path: ['confirmPassword'] }
  );

export type Step1Data = z.infer<typeof step1Schema>;
```

### 3.2 – useForm z resolverem Zod

```
import { useForm } from 'react-hook-form';
```

```
import { zodResolver } from '@hookform/resolvers/zod';

const {
  register, handleSubmit, formState: { errors, isSubmitting },
  setError, watch, control
} = useForm<Step1Data>({
  resolver: zodResolver(step1Schema),
  mode: 'onBlur',
  reValidateMode: 'onChange',
  defaultValues: { firstName: '', email: '', password: '', confirmPassword: '' }
});
```

### 3.3 – Controller dla checkboxów i pól kontrolowanych

```
<Controller
  name="notifications.email"
  control={control}
  render={({ field }) => (
    <input
      type="checkbox"
      id="notif-email"
      aria-label="Powiadomienia e-mail"
      checked={field.value}
      onChange={field.onChange}
    />
  )}
/>
```

### 3.4 – useFieldArray dla dynamicznych kategorii

```
const { fields, append, remove } = useFieldArray({
  control,
  name: 'categories',
  rules: {
    minLength: {
      value: 1,
      message: 'Wybierz co najmniej jedna kategorie'
    }
  }
});
```

### 3.5 – handleSubmit i obsługa błędów serwera

```
const onSubmit = async (data: FormData) => {
  try {
    await registerUser(data);
    router.push('/welcome');
  } catch (err) {
    if (err.status === 409) {
      // email zajęty - mapujemy błąd na pole formularza
      setError('email', {
        type: 'server',
        message: 'Ten adres e-mail jest już zarejestrowany',
      });
      goToStep(1);
    } else {
      setError('root.serverError', { message: 'Błąd serwera, spróbuj ponownie' });
    }
  }
};
```

## 4. Checklist dostępności (WCAG 2.1 AA)

Każda pozycja musi być zrealizowana w oddanym projekcie. Weryfikacja automatyczna:  
Lighthouse Accessibility Score  $\geq 90$ .

| Element           | Wymaganie                                                                                                    | Jak sprawdzić                        |
|-------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------|
| ARIA labels       | Każdy <input> powiązany z <label for=> lub aria-label. Nie używać wyłącznie placeholder.                     | Chrome DevTools → Accessibility Tree |
| aria-describedby  | <span role="alert"> z błędem + aria-describedby="field-err" na polu. Screen reader odczyta błąd natychmiast. | axe DevTools, Accessibility Tree     |
| aria-invalid      | Ustawiane na "true" programowo gdy errors[field] istnieje. Nie polegać wyłącznie na kolorze.                 | Inspektor elementów                  |
| aria-required     | HTML5 required lub aria-required="true" + gwiazdka (*) z legendą powyżej formularza.                         | Walidator W3C, axe                   |
| Focus management  | Po przejściu do kroku: focus przenosi się na nagłówek h2 lub pierwsze pole. Użyj useEffect + ref.focus().    | Nawigacja klawiszem Tab              |
| Focus ring        | Widoczny outline na wszystkich elementach interaktywnych. Kontrast min 3:1 (WCAG 2.4.11).                    | Tryb HCM, Lighthouse                 |
| Breadcrumb kroków | aria-current="step" na aktywnym kroku. Wizualne wskaźniki mają tekstowe odpowiedniki.                        | Screen reader NVDA/VoiceOver         |
| Kontrast tekstu   | Min 4.5:1 AA dla normalnego tekstu, 3:1 dla dużego. Dotyczy placeholderów i błędów.                          | WebAIM Contrast Checker              |
| Stan ładowania    | disabled + aria-busy="true" + zmiana tekstu przycisku podczas isSubmitting.                                  | Inspektor, screen reader             |

## 5. Szkielet kodu do uzupełnienia

Fragменты oznaczone // TODO student uzupełnia samodzielnie zgodnie z instrukcją z sekcji 3.

### MultiStepForm.tsx – komponent nadrzędny

```
import React, { useState, useRef, useEffect } from 'react';
import { Step1 } from './Step1';
import { Step2 } from './Step2';
import { Step3 } from './Step3';
import { Step1Data, Step2Data } from './schemas';

export default function MultiStepForm() {
  const [currentStep, setCurrentStep] = useState(1);
  const [formData, setFormData] = useState<{ step1?: Step1Data; step2?: Step2Data }>({});
  const headingRef = useRef<HTMLHeadingElement>(null);

  useEffect(() => {
    // TODO: przenieś focus na headingRef po zmianie kroku
  }, [currentStep]);

  const handleStep1Complete = (data: Step1Data) => {
    // TODO: zapisz dane kroku 1 i przejdź do kroku 2
  };

  return (
    <main aria-label="Formularz rejestracji">
```

```

<nav aria-label="Postęp rejestracji">
  { /* TODO: breadcrumb z aria-current="step" */ }
</nav>
<h2 tabIndex={-1} ref={headingRef}>
  { /* TODO: tytuł aktualnego kroku */ }
</h2>
{currentStep === 1 && <Step1 onComplete={handleStep1Complete} />}
{currentStep === 2 && <Step2 ... />}
{currentStep === 3 && <Step3 data={formData} ... />}
</main>
);
}

```

### Step1.tsx – fragment pola password z siłą hasła

```

const passwordValue = watch('password');
const getPasswordStrength = (pwd: string) => {
  // TODO: zwróć 'słabe' | 'średnie' | 'silne'
  // Wskazówka: sprawdź długość, /[A-Z]/, /[0-9]/, /^[A-Za-z0-9]/
};

<label htmlFor="password">Hasło *</label>
<input
  id="password"
  type="password"
  aria-required="true"
  aria-invalid={!errors.password}
  aria-describedby={errors.password ? 'pwd-err' : 'pwd-hint'}
  {...register('password')}
/>
<span id="pwd-hint" aria-live="polite">
  { /* TODO: wskaźnik siły hasła */ }
</span>
{errors.password && (
  <span id="pwd-err" role="alert">{errors.password.message}</span>
)}

```

## 6. Anti-patterns w formularzach

### Nazwa

|                             |                                                                               |                                                                                                 |
|-----------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Walidacja tylko przy submit | <b>X Źle</b><br>mode: 'onSubmit'                                              | <b>✓ Dobrze</b><br>mode: 'onBlur',<br>reValidateMode: 'onChange'                                |
| Brak powiązania label-input | <b>X Źle</b><br><span>E-mail</span> <input type="email" />                    | <b>✓ Dobrze</b><br><label htmlFor="email">E-mail</label> <input id="email" type="email" />      |
| Stan poza RHF               | <b>X Źle</b><br>const [val, setVal] = useState('') // podwójne źródło prawdy! | <b>✓ Dobrze</b><br>const { register } = useForm() // RHF = jedyne źródło prawdy                 |
| reset() przy cofnięciu      | <b>X Źle</b><br>// kasuje dane kroku!<br>reset();                             | <b>✓ Dobrze</b><br>// przechowaj w stanie nadrzędnym useForm({ defaultValues: formData.step1 }) |

|                                   |                                                                                               |                                                                                                                                                                               |
|-----------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Blokowanie Submit bez info</b> | <b>X Źle</b><br><code>&lt;button<br/>disabled={!isValid}&gt;<br/>Dalej &lt;/button&gt;</code> | <b>✓ Dobrze</b><br><code>&lt;button<br/>disabled={isSubmitting}<br/>aria-busy={isSubmitting}&gt;<br/>{isSubmitting ?<br/>'Wysyłanie...' : 'Dalej'}<br/>&lt;/button&gt;</code> |
|-----------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7. Kryteria oceny

Ocena wystawiana zgodnie ze skalą AT: 5.0 / 4.5 / 4.0 / 3.5 / 3.0 / 2.0. Brak zaliczenia przy Lighthouse Accessibility < 70 lub braku działającego submitu.

| Kryterium              | Opis wymagań                                                                                   | Waga |
|------------------------|------------------------------------------------------------------------------------------------|------|
| Działanie formularza   | Poprawne przejścia 1→2→3 i cofanie, dane nie giną przy cofnięciu, submit wysyła kompletne dane | 25%  |
| Walidacja Zod          | Schematy dla 3 kroków, refine(), polskojęzyczne komunikaty, typowanie przez z.infer<>          | 25%  |
| React Hook Form API    | Użycie register, Controller, useFieldArray, setError, watch. Brak duplikowania stanu poza RHF. | 20%  |
| Dostępność WCAG        | Lighthouse Accessibility ≥ 90, aria-labels, aria-invalid, aria-describedby, focus management   | 20%  |
| Obsługa błędów serwera | Błąd 409 mapowany na pole email + powrót do kroku 1. Generyczny błąd przez root.serverError.   | 10%  |

### Literatura i zasoby

- Dokumentacja React Hook Form: [react-hook-form.com](https://react-hook-form.com)
- Dokumentacja Zod: [zod.dev](https://zod.dev)
- WCAG 2.1 Quick Reference: [w3.org/WAI/WCAG21/quickref](https://w3.org/WAI/WCAG21/quickref)
- WebAIM Contrast Checker: [webaim.org/resources/contrastchecker](https://webaim.org/resources/contrastchecker)
- Lighthouse: wbudowany w Chrome DevTools (zakładka Lighthouse)
- axe DevTools: rozszerzenie Chrome/Firefox